Sagar Singh Bhola

734-905-2489 | sagbho@umich.edu | LinkedIn | GitHub | Portfolio | **U.S. & Canadian Citizen**

Education

University of Michigan, Ann Arbor — December 2025
Bachelor of Science in Computer Science — *GPA: 3.75*

- **Relevant Coursework:** Data Structures and Algorithms, Computer Networks, Computer Security, Web Systems, Computer Architecture, User Interface Development, Human-Centered Software

Technical Skills

Languages/Technologies: Python, C++, C, JavaScript, TypeScript, SQL, Java, HTML, CSS
Frameworks/Libraries: React, Next.js, Vue, Node.js, Express.js, Flask, FastAPI, Pandas, PyTorch, OpenCV
Tools/Platforms: MySQL, Docker, AWS, Git, Postman, Playwright, Figma

Experience

Data Engineering Co-op — May 2025 – Present
Ciena Healthcare — *Southfield, MI*

- Engineered automated preparation and validation workflows for Michigan Medicaid cost reporting by standardizing and populating import templates with operational and financial data across 51 nursing facilities, supporting an 84% first-pass acceptance rate for 2025 filings and helping avoid approximately $100K in external preparation costs.
- Built a reusable Python ETL pipeline with Pandas, pdfplumber, regex, and OpenPyXL to extract and transform 2,000+ Medicaid reimbursement-rate PDFs across 51 facilities and four fiscal years, reducing a multi-day manual process to a historical backfill completed in under six hours.
- Developed recurring Python and SQL data reconciliation pipelines to compare PointClickCare census and patient-in-bed data with general-ledger records across 51 facilities, automatically identifying cross-system discrepancies prior to monthly financial close.

Projects

Vision-Based Autonomous Driving Agent | *Python, C++, PyTorch, OpenCV, Pandas, DXcam*

- Developed and evaluated a closed-loop behavior-cloning agent for Assetto Corsa that learned steering, throttle, and braking directly from first-person visual input, autonomously navigating highway traffic without human control.
- Designed an end-to-end computer-vision training pipeline that synchronized 80,000+ gameplay frames with Thrustmaster T300RS steering, throttle, and brake inputs across 15+ driving sessions, using session-level train/validation separation and automated data-quality validation to reduce temporal leakage.
- Built a real-time perception-to-control system integrating 30 FPS visual capture, GPU-accelerated neural-network inference, virtual pedal actuation, and physical steering through a C++ force-feedback control layer for the Thrustmaster T300RS.

AI Invoice Processor | *Next.js, TypeScript, FastAPI, Python, SQLite, Local LLM*

- Developed a privacy-focused invoice processing platform that extracted invoice headers, line items, vendor information, and accounting distributions from uploaded PDFs using a locally hosted LLM, keeping 100% of document data on-device while processing 50+ invoices with an extraction accuracy of 92%.
- Engineered an end-to-end document processing pipeline using Next.js, FastAPI, Python, and SQLite to manage file ingestion, structured extraction, validation, review-state transitions, and accounting records, reducing average invoice entry time from 4 minutes to 1 minute.
- Built an accounts-payable review dashboard with status-based work queues, embedded PDF previews, editable extracted fields, automatic draft persistence, and role-aware configuration for vendors, facilities, general-ledger accounts, and authorized users, enabling corrections without reprocessing source documents.

YesNoFingers | *Python, OpenAI Whisper, AWS Translate, AWS EC2, REST APIs*

- Developed a cloud-based speech translation system in partnership with the University of Michigan Language Resource Center to transcribe and translate culturally specific medical conversations across 40+ languages using OpenAI Whisper, AWS Translate, and team-curated terminology datasets.
- Engineered a Python-based processing pipeline on AWS EC2 that transformed recorded speech into translated text through audio validation, transcription, terminology-aware preprocessing, translation, and structured API responses, achieving an average end-to-end processing time of 3 seconds per recording.
- Improved system security and translation reliability through input validation, API security testing, failure-handling safeguards, and end-to-end workflow testing, incorporating feedback from 5 clinicians during pilot deployment at a Detroit-area free clinic to strengthen request handling and privacy-conscious processing.